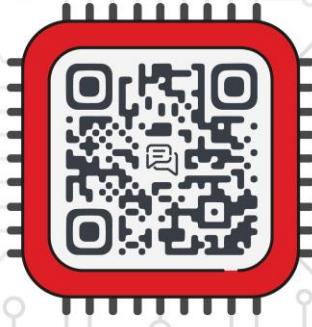




4



12



1440 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

## ملحق تمارين محاضرة شجرة البحث الثنائي

د. عملي مشترك



RB Informatics; 11/05/2025

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

سنداول ضمن المحاضرة كتابة بعض التمارين بطريقة تكرارية بدلاً من العودية بحيث يمكن للطلاب تصور متى من الأسهل استخدام التكرار ومتى من الأسهل استخدام التوابع العودية (الموضوع كيفي بالطلق وهذا التنويع للفائدة)

### التمرين الأول:

لتكن لدينا شجرة بحث ثنائي والمطلوب:  
أثبت أن عقدة ما ولتكن  $a$  سلف لعقدة أخرى ولتكن  $b$

تذكرة: السلف لعقدة  $b$  يمكن أن يكون أي عقدة على المسار من  $b$  وحتى الجذر (يشمل ذلك العقدة  $b$  نفسها)

#### فكرة الحل:

#### بداية

نقوم بالتأكد من وجود  $b$  ضمن الشجرة، يمكن تأجيل هذه العملية لوقت لاحق لكن لا إشكال في فعل ذلك ابتداءً، الهدف من ذلك سنذكره في النهاية.

#### ثانياً

عند كل عقدة نقف عندها في الشجرة يجب أن نتحقق أن  $a$  و  $b$  في نفس الجهة بالنسبة لهذه العقدة (في طرف ابنها الأيمن أو في طرف ابنها الأيسر) وهذا استفادة من خصائص البحث الثنائي، لماذا؟  
لأن العقدة لو كانت في فرع مختلف عن العقدة  $b$  لا يمكن أن تكون سلفاً لها، بمعنى لو كان لدينا العقدة  $x$  تقع  $a$  في شجرة ابنها الأيمن وتقع  $b$  في شجرة ابنها الأيسر لا يمكن ل  $a$  أن تكون سلفاً ل  $b$  لأنهما في طرفين مختلفين بالنسبة لعقدة ما.

في حال كانت العقدتان بنفس الجهة:

سنستمر بالذهاب نحو هذه الجهة الواحدة التي تحوي العقدتين.

إذا كانت كلا العقدتين يمين (أو أكبر من العقدة الحالية) نذهب للعقدة اليمين التالية، وإذا كانت كلتا العقدتين

يسار (أو أصغر من العقدة الحالية) نذهب للعقدة اليسار التالية.

## ثالثاً

نستمر حتى نصل لنقطة افتراق بالجهة وهذه النقطة قد تكون أحد الأمور الآتية:  
 إما أننا نجد a بحيث تصبح قيمة العقدة تساوي a وبالتالي يمكننا إرجاع true كون a تقع بنفس الجهة هي و b.  
 أو نجد b وبالتالي a تقع بعد b (باتجاه الأوراق) وليس قبلها (باتجاه الجذر) وبالتالي نرجع false.  
 أو نجد عقدة لا تساوي ال a ولا ال b بحيث تصبح قيمة العقدة تساوي عدداً لا يمكن من خلاله الانتقال لقيمة a في الشجرة بالتالي نرجع false.

## أخيراً

السبب من التحقق من b في البداية هو كوننا نتحقق من جهة وجود العقد باستخدام عمليات مقارنة للقيمة (أكبر/أصغر) وسنتوقف عندما نجد مفترقاً للطرق ولن نصل ل b بعد ذلك وهذا يجعلنا لا نضمن وجودها ضمن الشجرة بشكل حتمي.

## سؤال: لماذا لا نبحث عن a كذلك؟

لأننا لو وصلنا لمفترق طرق لو لم تكن قيمة العقدة عنده مساوية ل a سنرجع false في كل الأحوال، أما b فلن نصل لها في كل الحالات ولو عثرنا على a فرضاً عند المفترق وكانت b غير موجودة بالمثل فالجواب هنا false بالضرورة، كون b غير موجودة بالشجرة.  
 بالتالي قمنا باستبعاد هذه الحالة من المقام الأول من خلال البحث عنها وإيجادها ابتداءً (لو لم نقوم بذلك لافترضت الخوارزمية وجودها وأعطت true في الحالة التي سبق وذكرناها).

سنقوم بالتنقل بين العقد من خلال نمط تكراري:

## CODE

```
class BinaryTree {
    Node root;

    class Node {
        int value;
        Node left, right;

        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
}
```

1

```
/*insertion code "you can check the
previous lecture if you have any problems"*/
Node insert(Node root, int value) {
    if (root==null)
        return new Node(value);
    if (value<root.value)
        root.left = insert(root.left,value);
    else
        root.right =
            insert(root.right,value);
    return root;
}
```

2

3

```
boolean search(Node root, int
value) {
    while (root!=null) {
        if (value<root.value)
root=root.left;
        else if (value>root.value)
root=root.right;
        else return true;
    }
    return false;
}
```

4

```
boolean isAncestor(Node root, int a, int
b) {
    //making sure that b exists
    if (!search(root,b)) return false;
    //reference to track nodes
    Node current = root;
    while (current!=null) {
        //if both nodes are to the left to my root
        if (a<current.value && b<current.value) {
            current = current.left;
        }
        //if both nodes are to the right to my root
        else if (a>current.value &&
b>current.value) {
            current = current.right;
        }
        //any other case
        else {
            break;
        }
    }
    //we must find a value and this value must be a if so then
    we can say
    //a is an ancestor of b (true)
    //else (false)
    return current!=null && (current.value==a);
} }
```

5

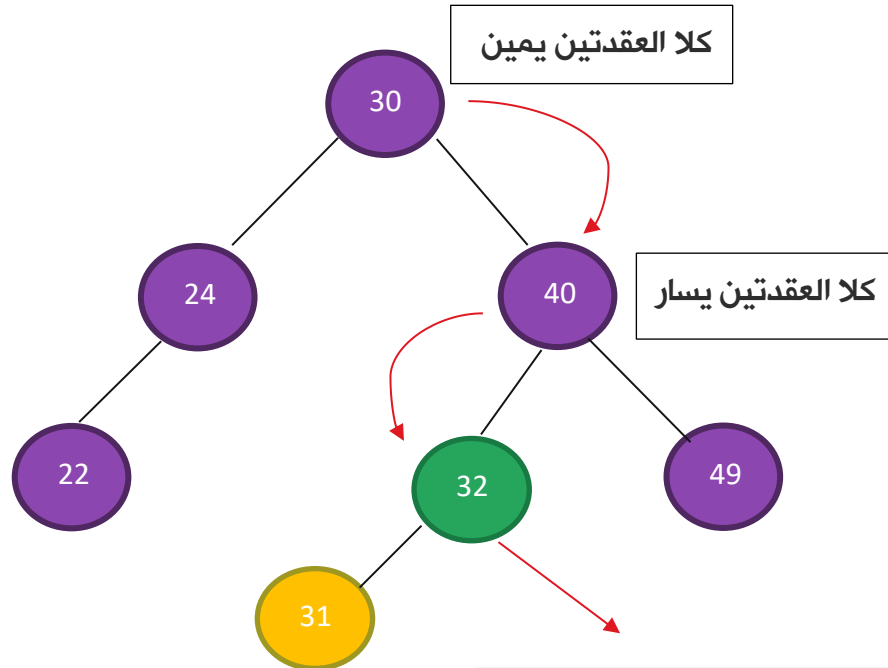
```
public class Main {
    public static void main(String[] args)
{
    BinaryTree bt = new BinaryTree();
    bt.root=null;
    int[] values = {50, 30, 70,
20,40,60, 80, 10,25, 35, 45,55,
65,75,85,5, 15, 33,37, 90};
    for (int val:values) {
        bt.root = bt.insert(bt.root, val);
    }

    System.out.println(bt.isAncestor(bt.ro
ot, 20,25)); //true
    System.out.println(bt.isAncestor(bt.ro
ot, 21,25)); //false
    System.out.println(bt.isAncestor(bt.ro
ot, 70,20)); //false
    }
}
```



## رسم للحالات التي ذكرناها في النهاية:

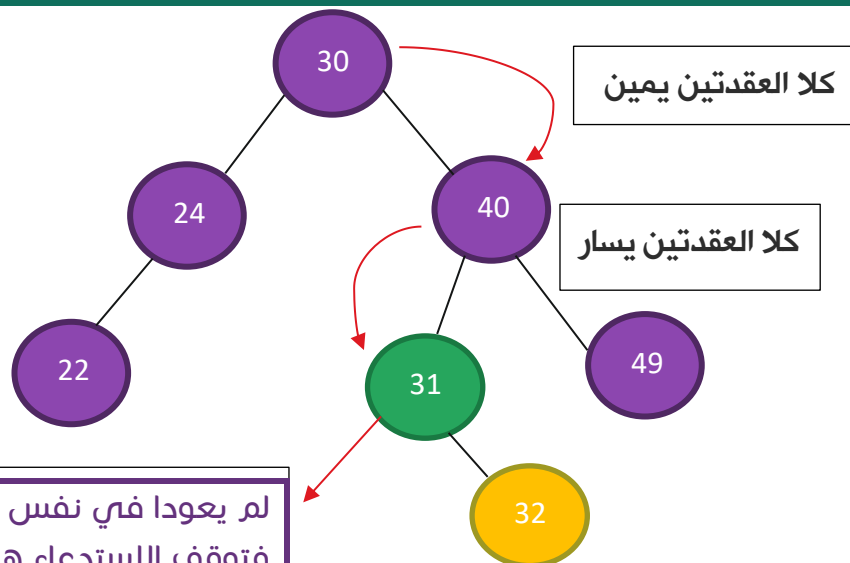
الحالة الأولى:



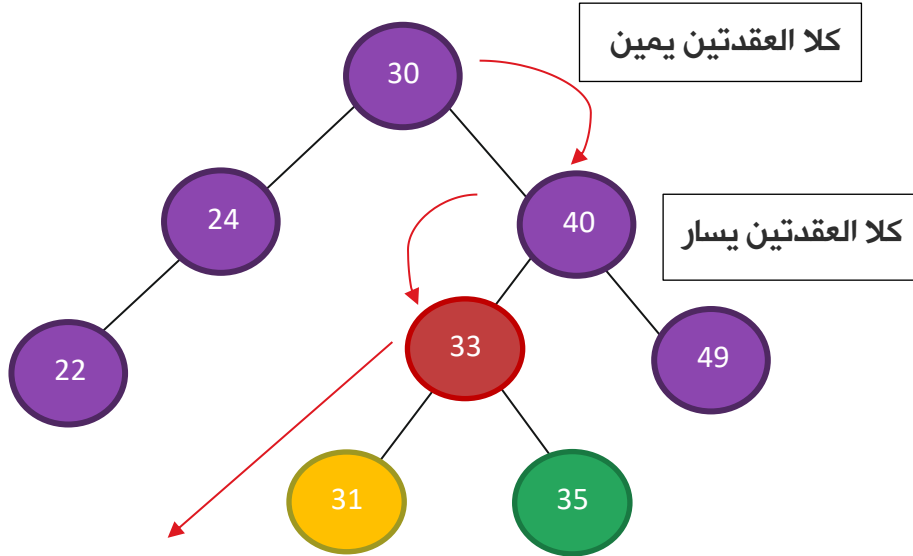
لم يعودا في نفس الجهة بالنسبة للعقدة  
فتوقف الاستدعاء هنا  
وفي هذه الحالة نتحقق إذا كنا قد عثرنا على  
A وفعلا هذه هي قيمة A فنرجع TRUE



الحالة الثانية:



لم يعودا في نفس الجهة بالنسبة للعقدة  
فتوقف الاستدعاء هنا  
وفي هذه الحالة نتحقق إذا كنا قد عثرنا على  
A ولكن بالواقع وجدنا B فنرجع FALSE



لم يعودا في نفس الجهة بالنسبة للعقدة فتوقف الاستدعاء هنا  
وفي هذه الحالة نتحقق إذا كنا قد عثرنا على A ولكن هذه القيمة هي قيمة لا تساوي لا A ولا B ولم يعد من الممكن أن تكون A في نفس مسار B فنرجع FALSE (كلا العقدتين في جهتين مختلفتين)



## التمرين الثاني:

ليكن لدينا شجرة بحث ثنائي المطلوب:  
أوجد السلف المشترك الأقرب لعقدتين معطائين a و b  
السلف المشترك الأقرب لعقدتين a, b: هو أقرب عقدة يلتقي فيها المسار من العقدة a حتى الجذر والمسار من العقدة b حتى الجذر

### فكرة الحل:

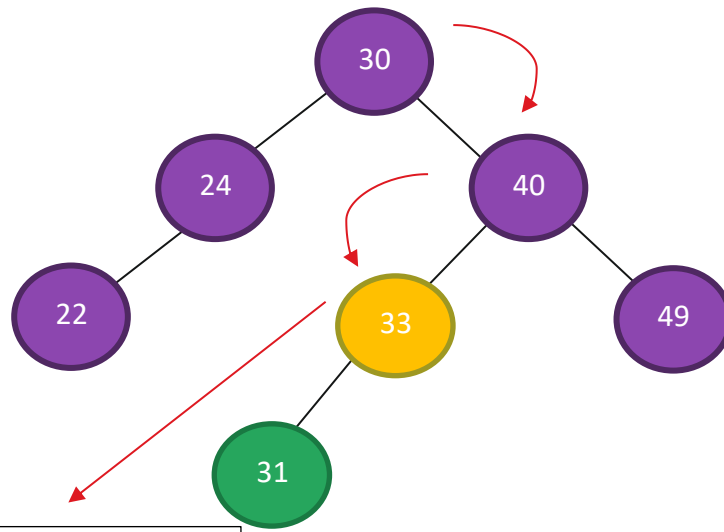
نقوم بالتأكد من وجود العقدتين اللتين سنبحث عن سلفهما المشترك الأقرب في حالة كانتا ليستا موجودتين نرجع قيمة حيادية -1 مثلاً.

ملاحظة: إذا كانت العقدتان موجودتان فمن المؤكد وجود سلف مشترك لهما أي لا يوجد حالة لا يكون فيها لهما سلف مشترك كون الجذر سلفاً لجميع العقد.

منطق الحل مشابه للمسألة السابقة فلن نفصل به كثيراً، نتحقق ما إذا كان  $a, b$  في جهة واحدة بالنسبة للعقدة الحالية فنستمر بالسير في هذه الجهة و نكمل هذا بالنسبة لكل عقدة نمر بها حتى نصل لنقطة افتراق (حتماً نقطة الافتراق هي أقرب سلف لكلا العقدتين فبعدها لا يمكن الاستمرار في أي جهة من الجهات).

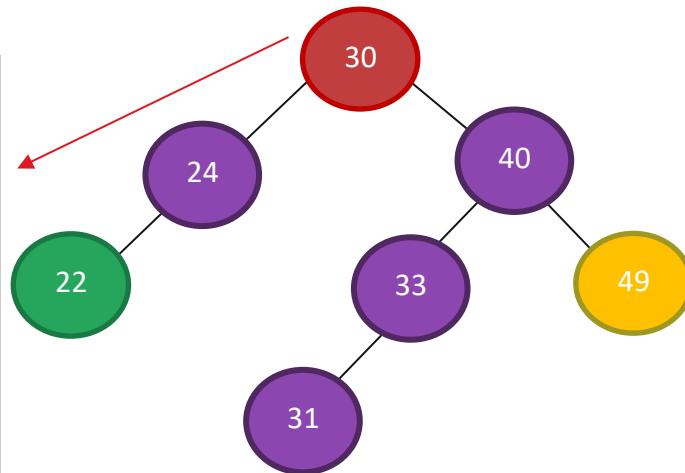
نقطة الافتراق قد تكون إحدى العقدتين المطلوبتين كذلك و لا إشكال في هذا إذ يمكن للعقدة أن تكون سلفاً لنفسها المختصر على عكس التمرين السابق لا يهمنا ماهية نقطة الافتراق فقط نبحت عن أبعد نقطة افتراق يمكننا الوصول لها و إرجاعها كإجابة.

#### ■ الحالة الأولى:



لم يعودا في نفس الجهة بالنسبة للعقدة فتوقف الاستدعاء هنا  
و تكون هذه العقدة هي السلف المشترك سواء كانت  $a$  أو  $b$  أو غير ذلك

#### ■ الحالة الثانية:



العقدتان ليستا في نفس الجهة بالنسبة للعقدة الحالية فتوقف الاستدعاء هنا  
و تكون هذه العقدة هي السلف المشترك سواء كانت  $a$  أو  $b$  أو غير ذلك

## CODE

```
class BinaryTree {
    Node root;
```

```
class Node {
    int value;
    Node left, right;
```

```
Node(int value) {
    this.value = value;
    this.left = this.right = null;
}
}
```

1

```
/*insertion code "you can check the
previous lecture if you have any
problems"*/
```

```
Node insert(Node root, int value)
{
    if (root==null)
        return new Node(value);
    if (value<root.value)
        root.left =
            insert(root.left,value);
    else
        root.right =
            insert(root.right,value);
    return root;
}
```

2

```
int lowestCommonAncestor(Node root,int
a,int b) {
    if (!search(root,a)||!search(root, b))
        return -1;

    while (root != null) {
        if (a< root.value &&b<root.value) {
            root= root.left;
        } else if (a
>root.value&&b>root.value) {
            root=root.right;
        } else {
            return root.value;
        }
    }
    return -1;
}
```

3

```
boolean search(Node root, int
value) {
    while (root!=null) {
        if (value<root.value)
            root=root.left;
        else if (value>root.value)
            root=root.right;
        else return true;
    }
    return false;
}
```

```

public class Main {
    public static void main(String[] args) {
        BinaryTree bt = new BinaryTree();
        bt.root=null;
        int[] values = {50, 30, 70, 20,40,60, 80, 10,25, 35, 45,55,
65,75,85,5, 15, 33,37, 90};
        for (int val:values) {
            bt.root = bt.insert(bt.root, val);
        }

        System.out.println(bt.lowestCommonAncestor(bt.root,20 ,10));
    }
}

```

### التمرين الثالث:

ليكن لديك شجرة بحث ثنائي اطبع القيم المحصورة ضمن مجال معين معطى [a,b] بشكل مرتب.

فائدة: عند طباعة شجرة البحث الثنائي باستخدام inorder traversal سنجد أن ما ينتج هو عبارة عن مصفوفة مرتبة.

### فكرة الحل:

بشكل عام يمكننا القيام بعملية inorder traversal طبيعية ووضع شرط قبل الطباعة إن كانت القيمة الحالية ضمن المجال فاطبعها

تذكرة: LEFT -> ROOT -> RIGHT = INORDER:

لكن يمكننا القيام بعملية تحسين على الاستدعاءات من ناحية التعقيد كون الشجرة هي شجرة بحث ثنائي:

✍ يمكننا الاستمرار بالنزول نحو اليسار حتى نصل إلى قيمة خارج المجال فنتوقف (لو كانت القيمة عند اليسار لا تزال

أكبر من بداية المجال فنستمر بالاستدعاء نحو اليسار) "يمكن لك تصغير القيمة طالما هي ضمن حدود المجال".

✍ تأتي مرحلة الطباعة الآن فنتحقق هل القيمة التي لدينا محصورة ضمن المجال لو كانت كذلك فنطبعها.

✍ ونأتي للخطوة الأخيرة، الذهاب نحو اليمين نستمر في النزول نحو اليمين حتى نصل لقيمة خارج المجال

(لو كانت القيمة عند اليمين لا تزال أصغر من نهاية المجال فنستمر بالاستدعاء نحو اليمين) "يمكن لك تكبير

القيمة طالما هي ضمن حدود المجال"

وبهذا سيقبل عدد الاستدعاءات الحاصلة ولن نتفقد جميع قيم الشجرة بل فقط القيم المشمولة ضمن المجال

تقريباً.

## CODE

```

class BinaryTree {
    Node root;
    class Node {
        int value;
        Node left, right;

        Node(int value) {
            this.value = value;
            this.left = this.right = null;
        }
    }
}

/*insertion code "you can check the previous
lecture if you have any problems"*/
Node insert(Node root, int value) {
    if (root==null)
        return new Node(value);
    if (value<root.value)
        root.left = insert(root.left,value);
    else
        root.right = insert(root.right,value);
    return root;
}

boolean search(Node root, int value)
{
    while (root!=null) {
        if (value<root.value)
            root=root.left;
        else if (value>root.value)
            root=root.right;
        else return true; }
    return false; }

```

```

void printBetween(Node root,
int a, int b) {
    if (root == null) return;
    //just make sure that a is
    the lower bound is a of the
    domain and b is the upper bound
    int min = Math.min(a,b);
    int max = Math.max(a,b);
    a= min;
    b = max;
    // we go left only when we
    have a value that's still larger
    than the left bound
    if (a<root.value)

    printBetween(root.left,a,b);
    //we print the value only
    when it's in the domain
    if(a<=root.value&&root.value<=b
    )
    System.out.print(root.value+" ");
    // we go right only when we
    have a value that's still smaller
    than the right bound
    if (b >root.value)
        printBetween(root.right,a,b);
    }
}

```

```
public class Main {  
    public static void main(String[] args) {  
        BinaryTree bt = new BinaryTree();  
        bt.root=null;  
        int[] values = {50, 30, 70, 20,40,60, 80, 10,25, 35, 45,55,  
65,75,85,5, 15, 33,37, 90};  
        for (int val:values) {  
            bt.root = bt.insert(bt.root, val);  
        }  
        bt.printBetween(bt.root,5 ,40);  
    }  
}
```

### التمرين الرابع:

ليكن لديك شجرة بحث ثنائي سيتم إعطاؤك قيمة  $x$  أوجد أقرب عدد موجود ضمن الشجرة للقيمة المعطاة.

#### فكرة الحل:

نقوم بعمل متغير لحفظ الإجابة "أقرب قيمة ل  $x$  ضمن الشجرة" ونحدثها عند كل عقدة.  
بعد المقارنة:

عند كل عقدة نقف عندها نتأكد هل هي أقرب للعدد الذي لدي من جميع العقد السابقة أم لا إن كانت كذلك نقوم بالاحتفاظ بالقيمة الجديدة  
(يعني هذا المتغير سنستخدمه لنقارن الفرق بين قيمة العقدة الحالية والعدد المعطى وقيمة أفضل عقدة سابقة وجدناها والعدد المعطى).

"نأخذ القيمة المطلقة للفرق بين القيمة الحالية للعقدة والعدد المعطى ونقارنه بالقيمة المطلقة للفرق بين أفضل عقدة سابقة وجدناها والقيمة المطلقة".

ونستفيد من خصائص شجرة البحث الثنائي بكوننا سنقوم بالاستدعاءات في جهة معلومة "لو كانت القيمة التي نبحث عنها أصغر نذهب لليسار وإن كانت أكبر نذهب لليمين".

وسنستمر في ذلك حتى نهاية الشجرة ثم سنقوم بإرجاع أقرب قيمة عثرنا عليها.

ملاحظة: لو وجدنا القيمة المعطاة ضمن الشجرة نرجعها كما هي.

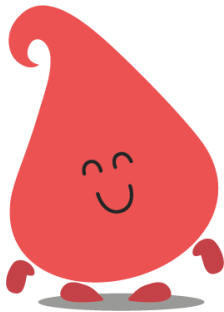
## CODE

```
class BinaryTree {  
    Node root;  
    class Node {  
        int value;  
        Node left, right;  
  
        Node(int value) {  
            this.value = value;  
            this.left = this.right = null;  
        }  
    }  
  
    /*insertion code "you can check the previous  
    lecture if you have any problems"*/  
    Node insert(Node root, int value) {  
        if (root==null)  
            return new Node(value);  
        if (value<root.value)  
            root.left = insert(root.left,value);  
        else  
            root.right = insert(root.right,value);  
        return root;  
    }  
    boolean search(Node root, int value)  
    {  
        while (root!=null) {  
            if (value<root.value)  
                root=root.left;  
            else if (value>root.value)  
                root=root.right;  
            else return true; }  
        return false; }  
}
```

```
int findClosest(Node root,int x)  
{  
    int best =root.value;  
  
    while (root != null) {  
        // check if we have better value  
        if (Math.abs(root.value-  
x)<Math.abs(best-x)) {  
            best =root.value;  
        }  
  
        //normal search  
        if (x<root.value) {  
            root = root.left;  
        } else if (x> root.value) {  
            root =root.right;  
        } else {  
            //if we find the exact value of a  
            node just return it  
            return root.value;  
        }  
    }  
    // else we will return the best  
    value so far  
    return best;  
}
```

4

```
public class Main {  
    public static void main(String[] args) {  
        BinaryTree bt = new BinaryTree();  
        bt.root=null;  
        int[] values = {50, 30, 70, 20,40,60, 80, 10,25, 35, 45,55, 65,75,85,5,  
15, 33,37, 90};  
        for (int val:values) {  
            bt.root = bt.insert(bt.root, val);  
        }  
        System.out.println(bt.findClosest(bt.root,82));  
    }  
}
```

**THE END...**